Student Task Manager

Project Report

Python Essentials Course

Submitted by: Riya Pal
Reg No: 25BOE10121
Branch: BIOENGINEERING
Year: FIRST YEAR
Course Code: CSE0001
Date: 30 March 2026

INTRODUCTION

This report documents the design, development, and learning outcomes of the Student Task Manager — a command line Python application built as part of the Python Essentials course on the VITyarthi platform.

The project was inspired by a real problem that almost every student faces: tasks are scattered across multiple places like WhatsApp messages, paper notebooks, sticky notes, and memory. There is no single organized system to track what needs to be done, what is already done, and what is overdue.

The Student Task Manager addresses this problem by providing a simple, menu-driven console application where students can add, view, prioritize, and manage their academic and personal tasks in one place.

PROBLEM STATEMENT

Observation

As a student, the following situation is very common:

- Assignment deadlines are saved in WhatsApp group chats and easily lost
- Exam dates are written in notebooks that may not always be at hand
- Project milestones exist only in memory and get forgotten
- There is no quick way to see what is most urgent right now

Pain Points

Pain Point	Impact
Forgetting low-priority tasks	They become urgent at the last minute
No daily focus	Students do not know what to work on today
No progress visibility	No sense of how much is pending vs. completed
Scattered information	Time wasted searching for task details

PROPOSED SOLUTION

The Student Task Manager is a menu-driven command line application that provides a centralized system for managing student tasks. The key design goals were:

- Simple to use - navigated entirely through numbered menu choices
- Useful for real student life - supports assignments, exams, projects, and personal tasks
- Persistent - tasks are saved to a JSON file, so data is not lost between sessions
- Modular - each feature is written as a separate Python module (file)

Features Implemented

#	Feature	Description
1	**Add Task**	Accepts title, category, priority, and due date from the user
2	**View All Tasks**	Displays all tasks in a formatted table with status icons
3	**Today's Tasks**	Filters and shows only tasks with today's due date
4	**Pending Tasks**	Lists incomplete tasks sorted by priority (High first)
5	**Mark as Done**	Updates the status of a selected task to Done
6	**Delete Task**	Permanently removes a task after confirmation
7	**Summary**	Shows overall stats, category breakdown, and priority bar chart

PROJECT STRUCTURE AND DESIGN

One of the key requirements of this project was to separate each feature into its own Python module (file). This approach is known as modular programming and is a fundamental software design principle.

File	Responsibility
main.py	Entry point — displays the menu and calls functions from all other modules
storage.py	Handles reading and writing tasks to the tasks.json file
add_task.py	Contains the add_task() function — collects task details from the user
view_tasks.py	Contains view_all_tasks(), view_todays_tasks(), view_pending_tasks()
mark_done.py	Contains the mark_done() function — updates task status
delete_task.py	Contains the delete_task() function — removes a task by ID
summary.py	Contains show_summary() — displays statistics and overview

Each module is imported into main.py using Python's import system:

from storage : import load_tasks

from add_task : import add_task

from view_tasks : import view_all_tasks, view_todays_tasks, view_pending_tasks

from mark_done : import mark_done

from delete_task : import delete_task

KEY DECISIONS MADE

JSON for Storage

Tasks are stored in a tasks.json file rather than a database or plain text file. JSON was chosen because it is human-readable, easy to work with using Python's built-in json module, and maps directly to Python dictionaries making it a natural fit for this project.

Menu-Driven Interface

A numbered menu interface was chosen over a command-line argument system because it is more beginner-friendly, easier to demonstrate, and requires no extra setup from the user. Anyone can run the program and navigate it without reading documentation.

Priority-Based Sorting for Pending Tasks

When viewing pending tasks, the list is sorted by priority (High → Medium → Low) instead of by creation date. This directly addresses the pain point of not knowing what to focus on-the most urgent tasks always appear at the top.

Confirmation Before Delete

A yes/no confirmation step was added before deleting a task. This prevents accidental deletion and improves the user experience, especially since there is no undo feature in a file-based system.

CHALLENGES FACED

Managing Task IDs After Deletion

When a task is deleted, the IDs of remaining tasks can become non-sequential (e.g., 1, 2, 4 after deleting task 3). This was accepted as a known limitation since re-numbering IDs would make the code more complex and could confuse users who remember their task IDs.

Date Handling

Handling dates required converting between two formats: the display format (DD-MM-YYYY) that is familiar to Indian users, and the storage format (YYYY-MM-DD) that allows correct alphabetical/chronological sorting. The datetime.strptime() and strftime() functions were used for this conversion.

Keeping Modules Independent

Since add_task.py and mark_done.py both need to save data, they both import save_tasks from storage.py. Similarly, mark_done.py and delete_task.py both show the task list, so they import view_all_tasks from view_tasks.py. Keeping these dependencies clean and avoiding circular imports required careful planning

PYTHON CONCEPTS DEMONSTRATED

Python Concept	Where It Is Used
Functions	Every feature is written as a function in its own module
Modules & Imports	main.py imports functions from 6 other modules
Lists	All tasks are stored in a Python list
Dictionaries	Each task is represented as a dictionary
File I/O (JSON)	storage.py reads/writes tasks.json using the json module
datetime module	Parsing, comparing, and formatting due dates
String Formatting (f-strings)	Displaying tasks in a neat, aligned table
Loops	Menu loop, iterating over tasks to display or search
Conditionals	Priority sorting, overdue detection, status checking
Exception Handling (try/except)	Handling invalid user input for task IDs and dates
List Comprehensions	Filtering pending tasks, today's tasks, and overdue tasks

WHAT I LEARNED

This project was a valuable learning experience that went beyond individual Python syntax. Some key takeaways include:

- Modular programming makes code easier to read, maintain, and debug. When a bug appears in one feature, you know exactly which file to look in.
- Real programs need to handle bad input gracefully. Adding try/except blocks and input validation made the program much more robust.
- Designing the data structure (the task dictionary) carefully at the start made all the other features easier to build.
- The datetime module is powerful but requires attention to format strings - a small typo can cause bugs that are hard to spot.
- Version control with Git is essential even for small projects. It allows you to track changes, experiment safely, and share your work professionally.
- Writing a good README is as important as writing good code. Documentation helps others (and your future self) understand and use the project.

CONCLUSION

The Student Task Manager successfully addresses the real problem of scattered and forgotten tasks that students face daily. Built entirely with Python's standard library, it demonstrates how even a simple program can be genuinely useful when it is designed around a real user need.

The modular structure of the project reflects good software engineering practices, and the use of file-based persistence makes it a practical tool that retains data across sessions. This project provided hands-on experience with core Python concepts including functions, modules, file I/O, data structures, and exception handling.

Overall, this project reinforced the idea that programming is most meaningful when it solves a real problem and that writing clean, organized, well-documented code is just as important as making the program work.